

Interactions or Text? Comparing Two Neural Recommenders for Books

Yuwen Peng

Department of Computational Linguistics

Heidelberg University, Germany

peng@cl.uni-heidelberg.de

1 Introduction

Recommender systems generally follow two main approaches. Collaborative filtering focuses on users’ past interactions with items, without considering the content of the items themselves. The basic idea is that users with similar preferences are likely to enjoy similar items. Content-based recommendation takes a different approach: it uses information about the items themselves. For books, this can naturally include the text of their descriptions. These two approaches capture different kinds of information and can therefore complement each other. However, they are less often compared directly in the same controlled setting.

This project compares collaborative filtering and content-based recommendation in a common experimental setting. The collaborative filtering model uses matrix factorization (Koren et al., 2009), while the content-based model uses a fine-tuned Sentence-BERT encoder (Reimers and Gurevych, 2019). I then further extend the content-based model with metadata to examine whether information such as author and publication year provides additional value beyond the book description.

The evaluation focuses on both the differences between the two recommendation approaches and the contribution of metadata to the content-based model. I compare their overall ranking performance and examine how it varies with the amount of rating information available for a book, particularly for books with few or no ratings. I also compare the diversity of their recommendations, since models with similar ranking performance may still recommend different sets of books. Finally, I examine whether adding metadata improves the content-based model.

2 Dataset

The data comes from the Goodreads dataset released by Wan and McAuley (2018). The romance

Genre	Romance
Users	10,000
Books	61,217
Ratings	284,126
Density	0.046%
Train / dev / test	198,256 / 42,935 / 42,935
Median ratings/book	1
Positive ratings	69.5%

Table 1: Statistics of the working set.

subset consists of two files: one contains book metadata, including descriptions used as the textual input for the book encoder, and one contains user ratings.

Although the dataset is restricted to a single genre, the romance subset remains large. After filtering, it still contains 565,489 users and close to 19 million ratings. Due to the available computational resources, a subset of 10,000 users is sampled, retaining all ratings associated with these users.

The filtering procedure retains only interactions with an explicit rating and books with an available description, which serves as textual input for SBERT. The two files are then aligned so that every retained rating refers to a retained book, and every retained book occurs in the ratings data.

Throughout the experiments, ratings of 4 or 5 are treated as positive feedback. This definition is used both as the training signal and to determine whether a book is relevant during evaluation.

For each user, the ratings are split into training, development, and test sets with a ratio of 70/15/15, ensuring that every user occurs in all three splits. Table 1 summarizes the statistics of the resulting working set.

3 Models

3.1 Matrix factorization

Following [Koren et al. \(2009\)](#), the collaborative filtering model learns a vector p_u for each user and a vector q_i for each book, together with a user bias b_u and a book bias b_i . The predicted rating is

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^\top q_i, \quad (1)$$

where μ is the average rating in the training data. The model is trained by minimizing the squared error on the observed ratings with L2 regularisation,

$$\sum_{(u,i) \in \mathcal{R}} (r_{ui} - \hat{r}_{ui})^2 + \lambda (p_u^2 + q_i^2 + b_u^2 + b_i^2), \quad (2)$$

where $\mathcal{R}$ is the set of observed ratings. Following the terminology used in recommender systems, I refer to this model as matrix factorization or SVD, although it is not the classical matrix decomposition.

3.2 Sentence-BERT

The content-based model encodes each book description with all-MiniLM-L6-v2, a Sentence-BERT model ([Reimers and Gurevych, 2019](#)). Due to the available computational resources, only the top two layers of the encoder are fine-tuned. A feed-forward layer then maps the description representation to a book vector. The user vector is computed as the average of the book vectors for which the user gave positive feedback in the training data. When computing the user vector for a particular user-book pair, the book in that pair is excluded from the average.

The score of a user-book pair is the dot product of the user and book vectors. The model is trained with positive and negative user-book pairs. Positive pairs are formed from books for which the user gave positive feedback, while negative examples are sampled from the collection. The model is trained using a logistic loss.

3.3 Sentence-BERT with metadata

I further extend the content-based model with metadata. The author is represented by a learned embedding, while the publication year is represented by a normalised value. These features are combined with the description representation before the feed-forward layer, so that the resulting book vector incorporates both textual information and metadata.

4 Evaluation

All models are evaluated in the same way. For each user, every book in the collection is scored, books that the user rated in the training data are excluded, and the remaining books are ranked. Let R_u denote the books that received positive feedback from user u in the test split, and let T_u^K denote the top K books in the resulting ranking. Users with no positive feedback in the test split are excluded, leaving 8,282 users. Each configuration is trained with three random seeds. I report the mean and standard deviation across the three runs, with the metrics averaged over the evaluated users.

4.1 Ranking quality

Precision@K and Recall@K measure how many relevant books occur in the top K ,

$$P@K = \frac{|T_u^K \cap R_u|}{K}, \quad R@K = \frac{|T_u^K \cap R_u|}{|R_u|}. \quad (3)$$

Both metrics treat all positions in the top K equally. But NDCG@K ([Järvelin and Kekäläinen, 2002](#)) also considers the position of each relevant book by assigning a lower weight to books appearing further down the ranking,

$$DCG@K = \sum_{j=1}^K \frac{\mathbb{I}[i_j \in R_u]}{\log_2(j+1)}, \quad (4)$$

where i_j is the book at position j . I normalise this value by the score of an ideal ranking,

$$NDCG@K = \frac{DCG@K}{\sum_{j=1}^{\min(K, |R_u|)} \frac{1}{\log_2(j+1)}}. \quad (5)$$

This gives a value between 0 and 1 and accounts for differences in the number of relevant books across users. NDCG is used as the main ranking metric, since it distinguishes between rankings that contain the same relevant books but place them at different positions. Precision, Recall, and NDCG are reported for $K \in \{5, 10, 20\}$.

4.2 Book popularity

The overall metrics do not show whether performance depends on how frequently a book has been rated. I therefore also report the same metrics for groups of books based on the number of ratings they receive in the training data. Books with no training ratings form the cold-start group. For these

books, the model has to make recommendations without any rating information from other users.

4.3 Coverage

Ranking metrics do not show how widely the recommended books are distributed across the collection. A model could achieve similar ranking scores while recommending a relatively small set of books to many users. I therefore measure coverage as the proportion of books that appear in at least one user’s top- K recommendations,

$$\text{Coverage@}K = \frac{|\bigcup_u T_u^K|}{|I|}, \quad (6)$$

where I is the set of all books.

5 Experiments

5.1 Matrix factorization versus Sentence-BERT

Table 2 shows that the two models have similar NDCG values. Matrix factorization is slightly higher at $K = 5$, while Sentence-BERT is slightly higher at $K = 20$. Neither difference is larger than the variation across seeds. The difference is clearer in recall. At $K = 20$, Sentence-BERT retrieves 0.122 of the relevant books, compared to 0.085 for matrix factorization, and its precision is also higher at every K . Thus, the two models behave differently even though their NDCG values are similar. Sentence-BERT finds more relevant books, but many of these books are lower in the ranking. Recall still counts these books, while NDCG gives more weight to books near the top. This raises the question of which books Sentence-BERT is able to find.

Training ratings	Matrix fact.	Sentence-BERT
0	0.000 $\pm$ 0.000	0.005 $\pm$ 0.000
1–2	0.000 $\pm$ 0.000	0.008 $\pm$ 0.002
3–5	0.000 $\pm$ 0.000	0.007 $\pm$ 0.002
6–10	0.000 $\pm$ 0.000	0.013 $\pm$ 0.002
11+	0.095 $\pm$ 0.001	0.117 $\pm$ 0.010

Table 3: Recall@20 for books grouped by the number of ratings they have in the training data, mean and standard deviation over three seeds.

Table 3 shows where this difference comes from. Matrix factorization never retrieves a book with

fewer than eleven training ratings, in any of the three runs. Its recall therefore comes entirely from the books with the most training ratings. Sentence-BERT performs better even for these books and also retrieves books from every other group, including books with no training ratings. The two models therefore behave differently depending on how much rating information is available for a book. Matrix factorization mainly works for books that have been rated many times, while Sentence-BERT can also recommend books with little or no rating information. This explains why the difference between the models is much larger when I look at individual groups than when I only look at the overall results.

The zero values for matrix factorization can be explained by how the model learns the book vectors. The vector q_i of a book is learned from the ratings that the book receives, and the median book in the training data has only one rating. With so little information, the learned vector stays close to zero because of the regularisation. The prediction is then mainly determined by $\mu + b_u + b_i$, with the book bias b_i having the main effect on the ranking between books. This means that books that are rated well in general are preferred, rather than books that are a good match for a particular user. The resulting ranking is therefore very similar across users, which also explains why the results change very little between runs.

If the ranking is almost the same for every user, only a small number of books can appear in the recommendations. The coverage results show this clearly (Table 4). Out of 61,217 books, matrix factorization recommends only 33 different books to the 8,282 users at $K = 20$, and only 12 at $K = 5$. Sentence-BERT recommends 5,061 different books at $K = 20$. Thus, the two models have similar NDCG values, but the recommendations are very different. Sentence-BERT selects different books for different users, while matrix factorization produces almost the same list for everyone. The overall ranking metrics do not show this difference.

5.2 The effect of metadata

The second comparison gives a clearer difference. While the two models in Table 2 have similar NDCG values, the two versions of the content-based model differ clearly. NDCG@20 increases from 0.068 to 0.101 and Recall@20 from 0.122 to 0.188 (Table 5). Both differences are several times larger than the variation across seeds. Adding these

	@5	@10	@20
<i>Matrix factorization</i>			
Precision	0.020 ± 0.001	0.014 ± 0.000	0.009 ± 0.000
Recall	0.055 ± 0.003	0.069 ± 0.002	0.085 ± 0.001
NDCG	0.055 ± 0.002	0.059 ± 0.001	0.064 ± 0.001
<i>Sentence-BERT</i>			
Precision	0.020 ± 0.004	0.016 ± 0.002	0.012 ± 0.001
Recall	0.058 ± 0.012	0.084 ± 0.010	0.122 ± 0.007
NDCG	0.048 ± 0.012	0.057 ± 0.012	0.068 ± 0.011

Table 2: Ranking quality on the test split, mean and standard deviation over three seeds.

	Matrix fact.	Sentence-BERT
@5	12 ± 0	$1,839 \pm 67$
@10	18 ± 0	$3,036 \pm 126$
@20	33 ± 4	$5,061 \pm 217$

Table 4: Number of distinct books recommended to at least one user, out of 61,217, mean and standard deviation over three seeds.

two metadata fields therefore has a larger effect on the results than the difference between matrix factorization and the basic Sentence-BERT model.

The improvement also depends on the number of training ratings. The frequently rated books improve from 0.117 to 0.166, while books with no training ratings improve from 0.005 to 0.033. The groups with one to two and three to five ratings also show clear improvements. The number of different books recommended by the model increases from 5,061 to 10,217. Thus, metadata does not only improve the books that already have a strong rating signal. It also helps the model recommend books for which very little or no rating information is available.

The author is one possible reason for this result. An author’s identity does not depend on how often a book was rated, so a book with no training ratings can still use information learned from other books by the same author. A book description does not provide this connection: two books by the same author will only have similar representations if their descriptions are also similar. The publication year may have a smaller effect on its own, since about a fifth of the books do not have a publication year.

	Sentence-BERT	+ metadata
NDCG@20	0.068 ± 0.011	0.101 ± 0.007
Recall@20	0.122 ± 0.007	0.188 ± 0.009
<i>Recall@20 by training ratings</i>		
0	0.005 ± 0.000	0.033 ± 0.002
1–2	0.008 ± 0.002	0.043 ± 0.002
3–5	0.007 ± 0.002	0.059 ± 0.001
6–10	0.013 ± 0.002	0.070 ± 0.005
11+	0.117 ± 0.010	0.166 ± 0.004
Distinct books	$5,061 \pm 217$	$10,217 \pm 127$

Table 5: The content-based model with and without metadata, mean and standard deviation over three seeds. The middle block gives Recall@20 for books grouped by the number of ratings they have in the training data.

Since the two metadata features were trained together, these results do not show how much each feature contributes separately. They only show that using the two features together improves the content-based model.

6 Conclusion

This project compared matrix factorization and a content-based model based on Sentence-BERT for book recommendation. Although the two models achieved similar NDCG values overall, their behaviour was very different on books with few ratings. Matrix factorization mainly recommended books with many training ratings and produced very low coverage, while Sentence-BERT was able to recommend books across all rating groups, in-

cluding books with no training ratings. This shows that overall ranking metrics alone do not fully capture the differences between the two approaches in a sparse recommendation setting.

Adding author and publication year to the content-based model further improved both ranking quality and coverage. The improvement was also clear for books with little or no rating information, suggesting that metadata is useful when the available rating signal is weak. Since the two metadata features were trained together, their individual contributions cannot be separated in this experiment.

Overall, the results suggest that the choice of book representation is particularly important when the rating data is sparse. Looking at performance across rating groups and at coverage provides additional information that is not visible in the overall ranking metrics alone.

Limitations

The results depend on how much data was used. The romance subset contains 565,489 users, of which 10,000 were sampled, and this sampling is one of the reasons why the interaction data is so sparse: the median book in the training split has only one rating. The poor performance of matrix factorization on books with few ratings should therefore not be taken as a general weakness of the method. With a larger sample, more users would rate each book, and the model could learn more useful book vectors from these additional ratings.

The two models also use different training objectives. Matrix factorization is trained to predict rating values, while the content-based model learns to distinguish positive user-book pairs from sampled negative pairs. The comparison is therefore between two complete recommendation systems, rather than a direct comparison of text-based information and rating information under the same training objective.

The collection contains several editions of the same title, and some of the relevant books in the test split have at least one such duplicate. When a model recommends one edition while the test split contains another, this counts as a miss, so the reported scores are lower than they would be if different editions of the same title were treated as the same book. Editions of the same book share a description and therefore receive very similar representations in the content-based models. This

makes it more likely that several editions of the same book appear in the same recommendation list. The effect is therefore stronger for the content-based models than for matrix factorization.

References

- Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446.
- Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37.
- Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992.
- Mengting Wan and Julian McAuley. 2018. Item recommendation on monotonic behavior chains. In *Proceedings of the 12th ACM Conference on Recommender Systems (RecSys)*, pages 86–94.

A Declaration of AI usage

Generative AI tools (ChatGPT and Claude Code) were used as auxiliary tools during this project.

During this project, AI tools were used for:

- Improving the wording and clarity of parts of the report.
- Explaining technical concepts and helping with technical problems.
- Supporting the reading and understanding of relevant literature.
- Helping with debugging and improving code documentation, including comments and the README.

AI tools were not used to define the research questions, choose the experimental design, or interpret the experimental results. All AI-generated content was reviewed and corrected by me. I remain fully responsible for the accuracy, integrity and final content of the submitted work.